



35th IEEE International Conference on
Collaborative Advances in Software &
Computing **November 10-13, 2025**

A-MARL: Agile Multi-Agent Reinforcement Learning for Soft Real-Time Task Scheduling in Edge Computing

Authors: Amin Avan, Akramul Azim, Qusay Mahmoud.



Introduction: (1/3)

- Soft real-time applications (SRTA)
 - significant processing demands.
 - execution on embedded devices.
 - offloading portions of workload to edge computing (EC).
- Task scheduling
 - Performance of SRTA depends on timeliness & accuracy of '**task schedules**'.
 - Task scheduling in EC is NP-hard
 - Optimizes conflicting objectives of edge-users & edge-servers.
 - Requires analysis of numerous EC component parameters.
 - Handles EC environment dynamics.
 - Heuristic- & meta-heuristic-based task scheduling algorithm struggle to find optimal/near-optimal '**task schedules**', while reinforcement learning (RL) is capable.



Introduction: (2/3)

- Edge computing
 - Edge-users can utilize idle edge-server resources.
 - sensors, actuators, cameras, smartphones, IoT devices, vehicles.
 - Edge-servers
 - routers, switches, single-board computers, system-on-chip devices, server machines.
 - located in base stations, access points, roadside units.
 - Edge-users & edge-servers
 - Geographically dispersed & distributed
 - Heterogeneous computational, RAM, storage
 - Lower, more reliable, predictable communication latency than cloud computing

Introduction: (3/3)

- Reinforcement learning (RL):
 - Learns an optimal policy by interacting with environment via actions & receiving rewards.
 - Models complex environments by encoding multiple factors into states.
 - Dynamically updates its policies as conditions change.
 - Prolonged learning-time in *single-agent RL* vs. multi-agent RL (MARL).
 - MARL:
 - effective for SRTAs in cyber-physical systems, finance, communication networks.

Related works: (1/2)

Name	Agent Arch	Pros	Cons	SRTA Compat.
DPDS [14]	Single	1) Age-of-Information prioritizes fresh data. 2) Task scheduling is formulated as a Constrained MDP. 3) Edge-server energy efficiency improves.	1) Ignores SRTA requirements of tasks. 2) Assumes a single edge server.	No
DOSA [24]	Single	1) Considers task cycles & task data sizes.	1) Ignores edge-server heterogeneity, remaining processing & memory capacities, SRTA task response time.	No
DCOM [25]	Single	1) Enhance edge-users QoS by reducing communication latency & energy usage of them.	1) Large state-action space & DAG-based task dependencies significantly increase RL agent's learning time.	No
[27-32]	Single	1) Considers communication latency & bandwidth. 2) Reduce task execution time via edge-server computational and memory capacities. 3) Reduce task execution time & power usage of edge-users.	1) Suffers from slow convergence. 2) Ignores SRTA requirements of tasks.	No

Related works: (2/2)

Name	Agent Arch	Pros	Cons	SRTA Compat.
[33]	Multi	1) Enhances average task execution time.	1) Suffers from slow convergence. 2) Ignores SRTA requirements of tasks.	No
[34]	Multi	1) Uses Approx-based multi-agent MDP w/ Proximal Policy Opt. (PPO).		No
[36]	Multi	1) Variational Recurrent Neural Network (VRNN) & Long Short-Term Memory (LSTM) are employed for low-overhead Op. & modeling task arrival dynamics, resp.		No
[38]	Multi	1) Reduce task processing delays. 2) Improve task success rate.		
[40]	Multi	1) Reduce task processing delays.		No



Problem statement:

- Prolonged learning times in single- & multi-agent RL methods stem from “exploration phase” caused by trial-&-error in state-action space.
 - hampers compatibility with SRTA task scheduling in EC.
- SOTA task-scheduling algorithms target general-purpose apps, while SRTA requirements are often overlooked.

Problem formulation:

- Goal: Maximize hit-ratio with minimal learning-time (runtime)
 - Maximize hit-ratio:
 - C1.1: Assign an SRTA task to an edge-server only if its response time < its absolute deadline.
 - C1.2: Task is allocated only if the edge-server has enough idle processing & memory utilizations.
 - Minimize RL-agents learning-time (runtime):
 - C2.1: Each SRTA task must be allocated to exactly one edge-server.
 - C2.2: An edge-server is available only when utilization across all resources remains below a predefined threshold.

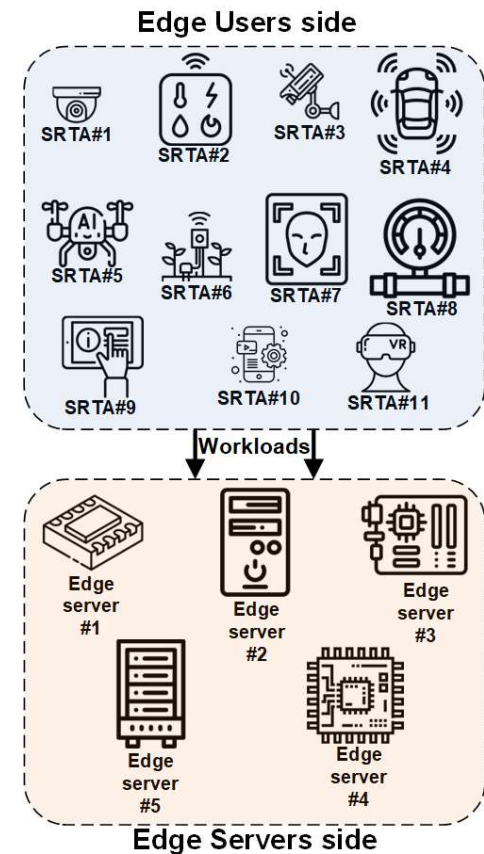


Fig.1. Edge computing (EC) system model.

Agile MARL (A-MARL): (1/2)

- Each action is a two-element tuple: [SRTA-task ID, edge-server ID].
- Each edge-server independently learns an optimal policy as an RL-agent in a shared environment.
 - Earn positive/negative rewards for satisfying/violating each constraint (C1.1-C2.2).
- Reducing A-MARL learning-time via ‘pseudo-action masking’ & ‘informed exploration’.
 - Pseudo-action masking
 - Actions conflicting with Shortest Processing Time (SPT) Alg are masked during high entropy.
 - Informed exploration
 - Selecting actions using SPT rather than relying on an immature policy under high entropy.

Agile MARL (A-MARL) – Workflow :

- Pseudo-Action Masking:
 - At the start of each episode, the policy is untrained & entropy remains high.
 - Action selection based on this policy is nearly random.
 - All possible state-action pairs are defined by MDP.
 - Action space pruning reduces state-action pairs to SPT's state-action space.
- Informed Exploration:
 - When entropy exceeds the threshold (Eq. 14), the RL-agent uses SPT to select actions in the state-action space under C1.2 & C2.2.

$$H_{th} = \alpha \cdot \log_e(|\cup_{(\tau \in \delta \epsilon \epsilon)} \tau| + 1) \quad (14)$$

Algorithm 1: Agile MARL (A-MARL)

Require: H_{th} , $\mathcal{T} = \{\tau_1, \tau_2, \dots, \tau_n\}$, $i \in \{1, \dots, N_{agent}\}$, θ_i^π , θ_i^V , $S = \{s_1, s_2, \dots, s_m\}$, N_{env} .
Ensure: $(\theta_i^{\pi*}, \theta_i^{V*})$

```

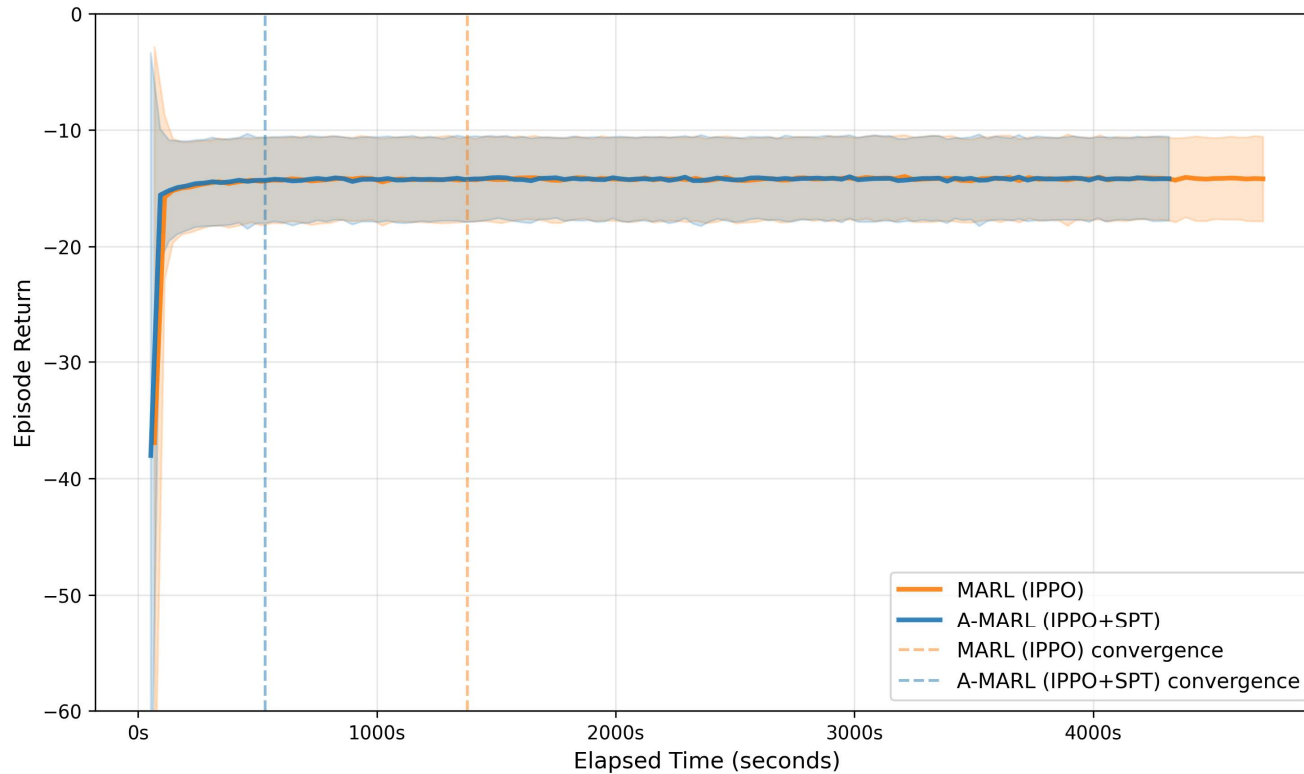
1: function GetSptAction( $obs_{agent}, agent_{id}, n_{actions}$ ) :
2:   # Each agent (edge-server) selects a task
3:    $features \leftarrow \text{ExtractFeatures}(obs_{agent}, agent_{id})$ 
4:    $available\_jobs \leftarrow \text{GetAvailableJobs}(features)$ 
5:    $spt\_job \leftarrow \text{ArgMin}(durations)$ 
6:    $action \leftarrow \text{MapJobToAction}(spt\_job)$ 
7:   Return  $action$ 
8: function
  SelectActionForAgent( $obs_{agent}, \pi_{\theta_i}, agent_{id}, step$ ) :
9:    $policy\_dist \leftarrow \pi_{\theta_i}(obs_{agent})$ 
10:   $a_{policy} \sim policy\_dist$ 
11:   $H_{\pi_i} \leftarrow \text{Entropy}(policy\_dist)$ 
12:   $H_{th} \leftarrow \text{Eq. (14)}$ 
13:  if ( $H_{\pi_i} > H_{th}$ ) then
14:     $a_{SPT} \leftarrow \text{GetSptAction}(obs_{agent}, agent_{id}, n_{actions})$ 
15:    Return  $a_{SPT}$ 
16:  else
17:    Return  $a_{policy}$ 
18: Main:
19: for (each agent  $i$ ) do initialize  $\pi_{\theta_i}$  and  $V_{\phi_i}$  end for
20: initialize  $N_{env}$ 
21: for ( $update = 1$  to  $n_{updates}$ ) do
22:   for ( $t = 1$  to  $T$ ) do # Collect rollout
23:     for each agent  $i \in \{1, \dots, N_{agents}\}$  do
24:       ( $a_i, t$ )  $\leftarrow$ 
        SelectActionForAgent( $obs_i, t, \pi_{\theta_i}, i, global\_step$ )
25:     end for
26:     Execute joint action  $a_t \leftarrow (a_{(1,t)}, a_{(2,t)}, \dots, a_{(N_{agents},t)})$ 
27:     Store transition  $(s_t, a_t, r_t, s_{t+1})$ 
28:   end for
29:   for each agent  $i \in \{1, \dots, N_{agents}\}$  do
30:     Compute advantages  $\hat{A}_i$  using GAE
31:     Update  $(\theta_i^\pi, \theta_i^V)$  using PPO for  $\mathcal{K}$  epochs
32:   end for
33:   Log agent-level exploration and entropy metrics
34: end for
35: Return  $\{\theta_1, \dots, \theta_{N_{agents}}\}$ 
36: end main

```

Experimental results:

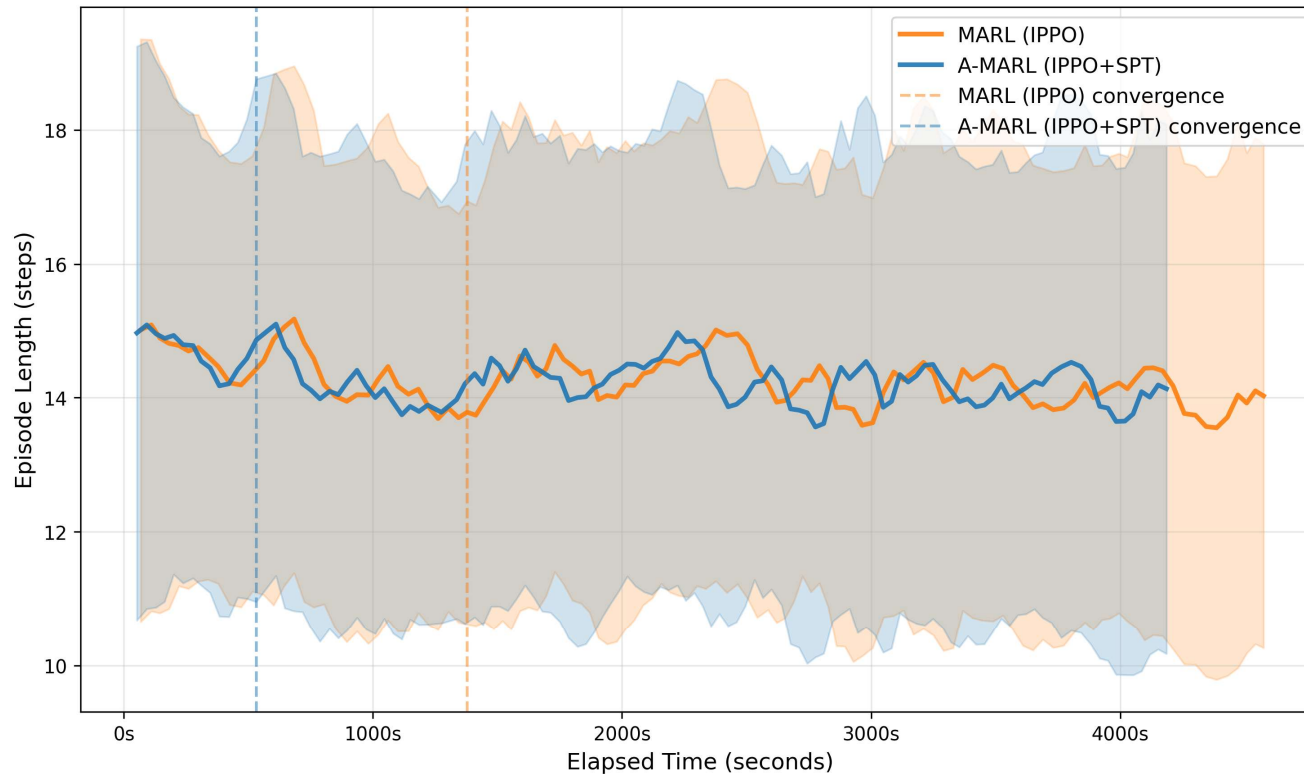
- Comparing two scheduling algorithms in solving Job-Shop problem [44].
 - standard MARL
 - A-MARL
- Job-Shop environment from Jumanji library is integrated into Mava framework.
 - Jumanji Job-Shop implementation was originally designed for single-agent RL.
- All experiments were performed on a Lenovo ThinkPad with an Intel® Core™ i5-8350U CPU (1.70-1.90 GHz), 16 GB RAM, and a 64-bit Windows OS on x64 architecture.
- Both methods are evaluated by
 - Solution quality
 - Convergence time
 - Computational efficiency

Experimental results – solution quality:



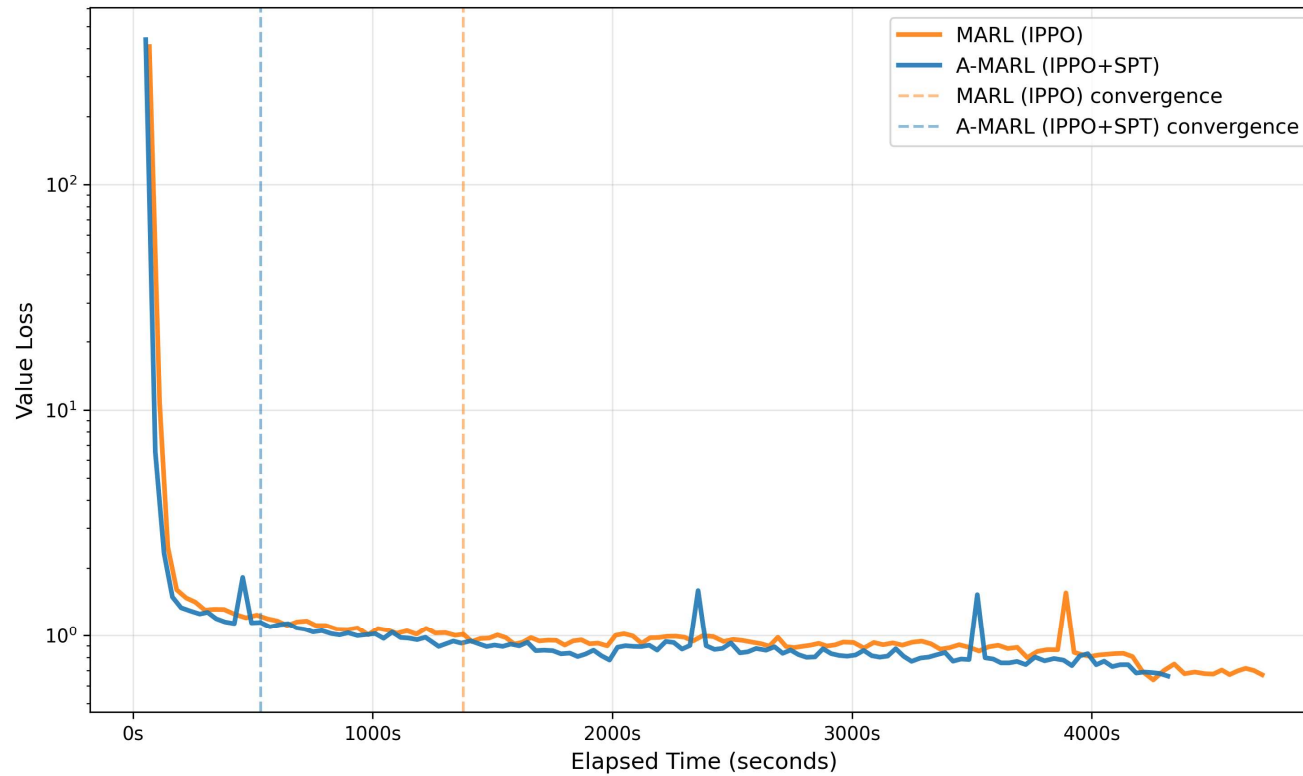
- A-MARL converges 61% faster than standard MARL.

Experimental results – learning dynamics:



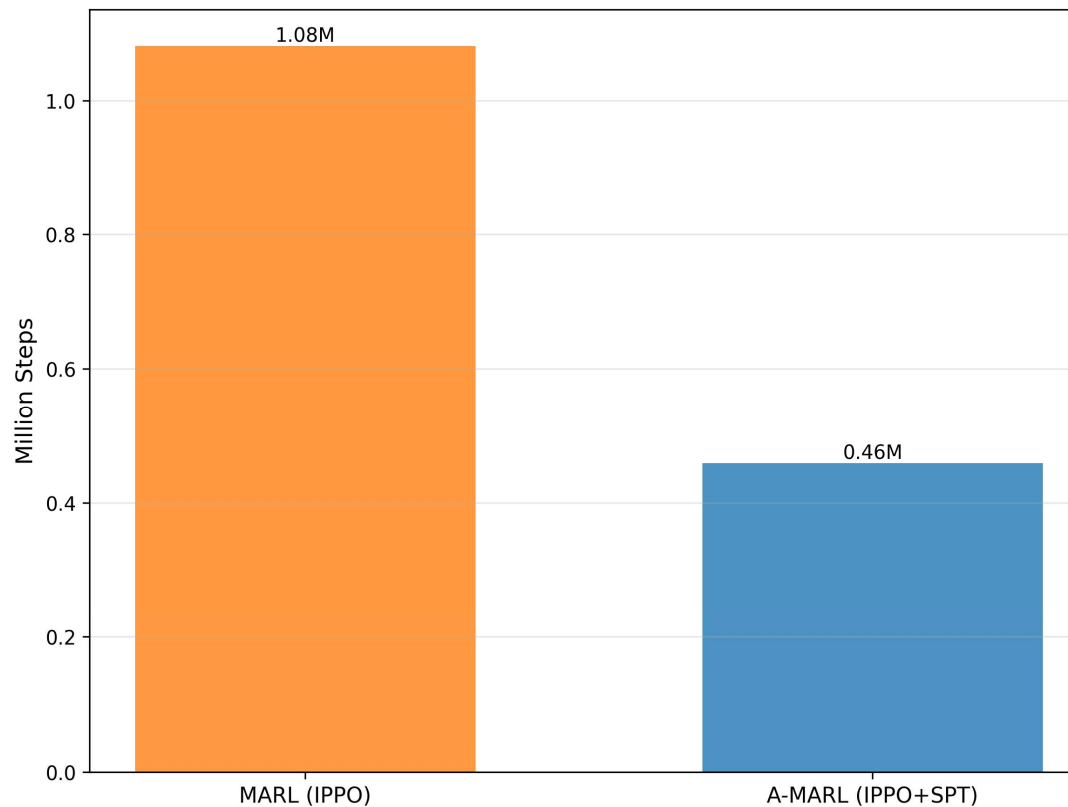
- While A-MARL maintains comparable episode lengths to standard MARL, its superior action selection quality enables faster convergence.

Experimental results – learning dynamics:



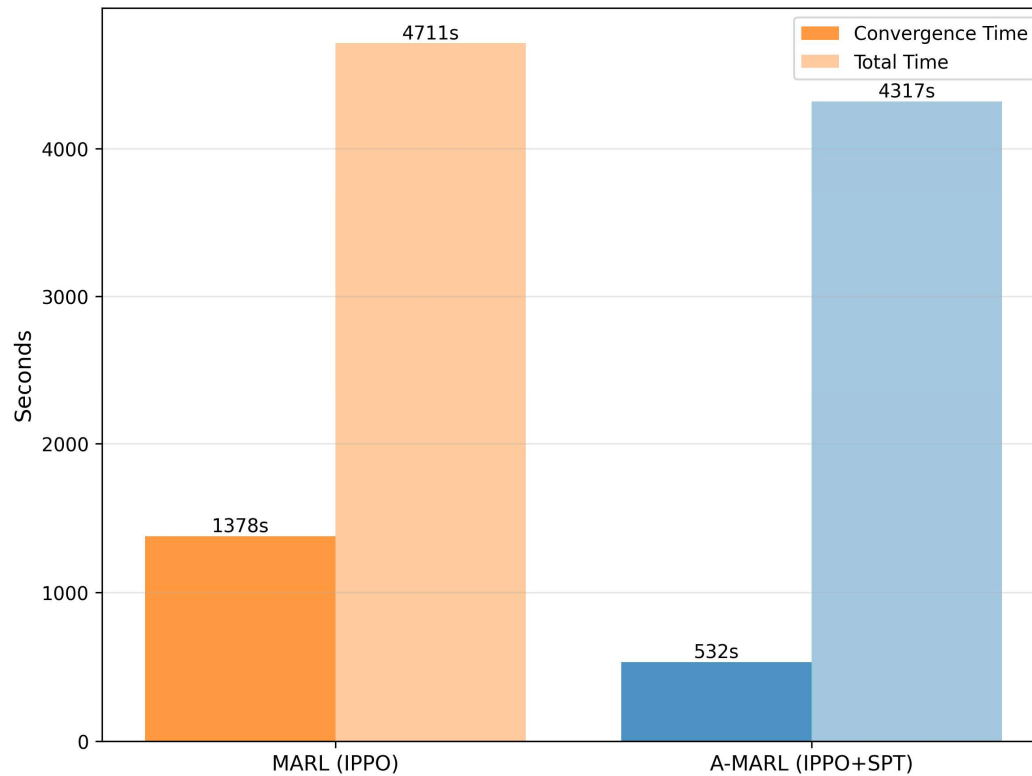
- A-MARL maintains lower value loss than standard MARL during learning-time.

Experimental results – computational efficiency:



- A-MARL requires 57% fewer steps than standard MARL for convergence.

Experimental results – computational efficiency:



- A-MARL converges 61% faster than standard MARL in seconds.



Conclusion:

- A-MARL tackles SRTA task scheduling in EC with reduced learning-time.
 - Achieved by incorporating ‘informed exploration’ & ‘pseudo-action masking’.
- A-MARL shows improved performance over standard MARL across multiple metrics:
 - Converges in 61% fewer seconds than standard MARL.
 - Lower value loss than standard MARL.
 - Converges in 57% fewer steps than standard MARL.